Level Up Computer Architecture Courses: Native RISC-V Coding with PRG32

Simone Boscaglia*, Ivan Cafiero*, Raffaele Montella*
*Department of Science and Technology, University of Naples Parthenope, Naples, Italy
{simone.boscaglia001, ivan.cafiero001}@studenti.uniparthenope.it,
raffaele.montella@uniparthenope.it

***Abstract*—Computer architecture courses need an easily accessible platform for students to write and execute complete programs while observing real constraints. PRG32 is an open-source retro-gaming ecosystem that runs assembly- or C-authored RV32IMAC cartridges natively on an ESP32-C6 or on a QEMU-based desktop emulator and exposes a common game-loop interface with a deliberately small runtime contract. PRG32 also provides a distribution service for easy cartridge sharing and a built-in performance harness to record per-frame update, draw, and present timings at 1 µs resolution and export structured JSON for reproducible classroom experiments.**

***Index Terms*—RISC-V, computer architecture education, embedded systems, performance measurement, reproducibility**

I. Introduction and Motivation

Undergraduate architecture teaching currently suffers from a gap between developing an accurate mental model of what is taught and a lack of an easy-to-use learning platform. Current solutions span from instruction-level simulators (RARS, Venus) that do not expose real-time constraints to FPGA platforms such as Wildcat [1] that expose microarchitecture at the cost of a substantial tool-chain and hardware-learning overhead.

PRG32 fills the gap with an open-source retro-gaming development platform system that executes native RISC-V code [2] on production silicon rather than emulating a CPU. Retro games make students face hard constraints, such as latency, data movement, compute time, and throughput trade-offs, similar to the type of constraints found in scientific and high-performance computing, where RISC-V is also an active target [3], [4].

PRG32 contributes an integrated ecosystem with (1) native RV32IMAC cartridge execution and real peripheral timing; (2) a stable runtime shared by assembly and C with a simple game-loop programming model; (3) cartridge packaging and wireless distribution; (4) a QEMU build path for reproducibility; (5) a measurement pipeline that exports per-frame data for classroom and research experiments; and (6) a complete textbook with two parallel teaching tracks for assembly and C. In Fig. 1, we propose a workflow for students that makes full use of the entire ecosystem.

II. Approach

The PRG32 reference hardware is assembled from off-the-shelf components chosen to minimize cost. It features an ESP32-C6 with a 160 MHz RV32IMAC core and 512 KiB SRAM [5], connected to a 320×240 RGB565 display over

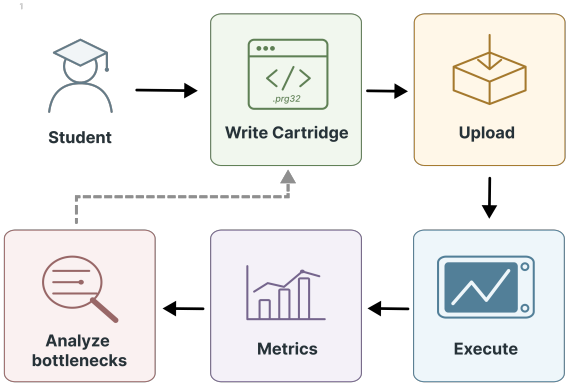


Fig. 1. Students write Assembly/C cartridges, deploy them to the Cartridge Store, execute on ESP32-C6 or QEMU, collect runtime metrics and iteratively optimize the implementation.

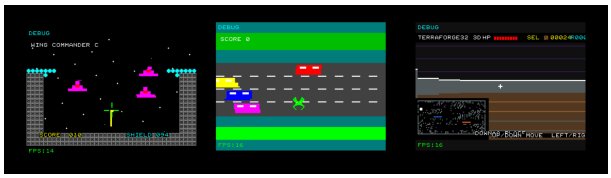


Fig. 2. Screenshots of three distinct cartridges: *Mineforge 3D*, alongside two others inspired by popular titles *Wing Commander* and *Frogger*. These and other cartridges are available for download on the Cartridge Store. Students can share their cartridges with other peers to increase engagement or to receive feedback.

40 MHz SPI, I²S audio, digital joysticks, and Wi-Fi. Every cartridge exports `init()`, `update()`, and `draw()`; after drawing, the runtime presents the framebuffer and pumps audio. The interface follows the RISC-V ABI and is shared by assembly and C `.prg32` cartridges. An ESP32-C3 build for Espressif's QEMU fork [6] is also available for students without a board but does not offer accurate timing.

The Flask/Docker Cartridge Store distributes architecture-tagged cartridges over Wi-Fi, such as those shown in Fig. 2, and also provides score, multiplayer-relay, and metrics services.

III. Examples of Active Learning

With PRG32, students can engage at different levels of complexity. Beginners can have fun writing simple games in low-level languages and making them work smoothly. To

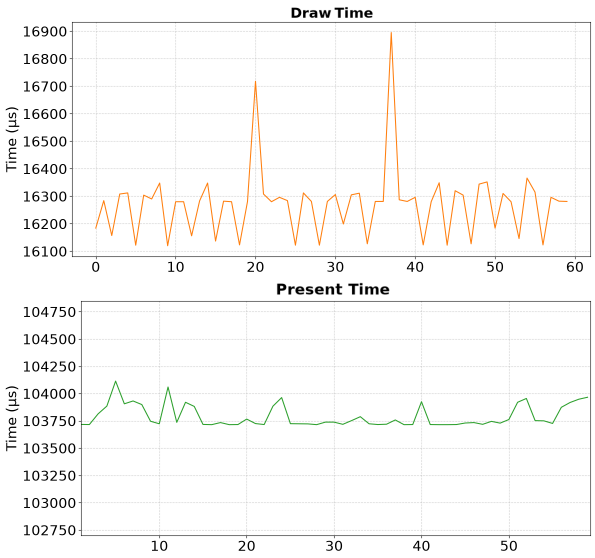


Fig. 3. Per-frame performance metric exports of draw time and present time in microseconds for a sample cartridge.

do this, they can use the performance tools integrated in the PRG32 ecosystem.

The firmware records key metrics for students to export: update, draw, present, free-heap, input, and deadline status for each frame; we show two example exports in Fig. 3. Export of per-frame data is uncommon among previous RISC-V platforms for education and allows students to examine how events line up with different frame-timings or compare assembly and C sources all within the runtime environment.

Students can analyze the spikes frame by frame and optimize their code to improve debugging skills and their overall understanding of computer systems. During gameplay, it is also possible to activate debug bands that display useful information, such as FPS, Wi-Fi connection, or custom strings.

Advanced students can look at the PRG32 code itself and modify it to observe how the changes impact the performance data. As an example activity, we propose a bottleneck analysis centered on modifying the SPI clock frequency. The SPI flash on the chip operates at a maximum clock frequency of 80 MHz; PRG32 uses a default value of 40 MHz. Students can lower the clock frequency to see how it impacts the frame times.

The metrics distinguish the Draw Phase, in which the CPU renders pixels into the frame buffer array in RAM, from the Present Phase, where the firmware hands off the RAM buffer address to the ESP32-C6's GDMA peripheral and triggers the SPI transfer. The Update Time is also included but cannot be seen because of its minimal overall impact on the frame times in this sample cartridge.

As seen in Fig. 4, by lowering the clock speed the present time increases, while the draw time remains constant. Students can observe how the draw time stays constant, learn about DMA, and understand that the CPU does not spend its own clock cycles continuously pushing bytes bit-by-bit over the SPI bus.

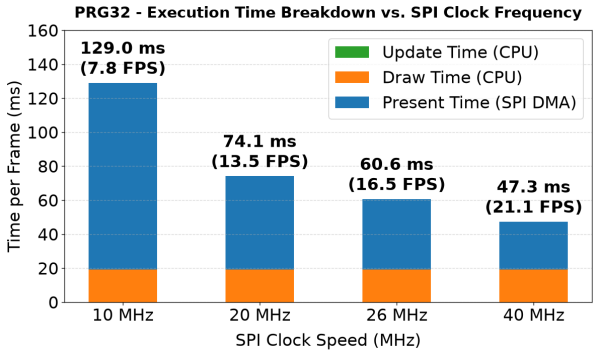


Fig. 4. Frame data is collected at 10 MHz, 20 MHz, 26 MHz, and 40 MHz because the available clock frequencies are obtained by dividing the maximum of 80 MHz by small integers. The update time is not visible because it is small compared to draw and present times.

IV. Conclusion and Future Work

PRG32 provides native RISC-V execution and observable peripheral costs with a cartridge sharing workflow and reproducible frame-level data, enabling students to observe and analyze hardware/software interactions beyond visual output alone. Next steps in the project roadmap include a controlled classroom pre/post study, assembly-vs-C comparative timing studies, and a runtime for Linux running on RISC-V boards without any microcontroller-specific toolchain.

Acknowledgments

This work is partially funded by the project "Environmental Awareness by the Sea and beYond (EASY)" (CUP I63C25000510007) and framed within the broader European effort towards RISC-V-based HPC sovereignty; the PRG32 firmware is open-source under the MIT license.

References

[1] M. Schoeberl, "Wildcat: Educational RISC-V microprocessors," in *Architecture of Computing Systems (ARCS 2025)*, ser. Lecture Notes in Computer Science, vol. 15839. Springer, 2025, pp. 189–202.

[2] K. Asanović and D. A. Patterson, "Instruction sets should be free: The case for RISC-V," EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2014-146, Aug. 2014. [Online]. Available: https://www2.eecs.berkeley.edu/Pubs/TechRpts/2014/EECS-2014-146.html

[3] W. Fornaciari, F. Reghenzani, F. Terraneo, D. Baroffio, C. Metra, M. Omaña, J. E. Rodriguez Condia, M. Sonza Reorda, R. Birke, I. Colonnelli, G. Mittone, M. Aldinucci, G. Mencagli, F. Iannone, F. Palombi, G. Zummo, D. Cesarini, and F. Tesser, "RISC-V-based platforms for HPC: Analyzing non-functional properties for future HPC and big-data clusters," in *Embedded Computer Systems: Architectures, Modeling, and Simulation – SAMOS 2023*, ser. Lecture Notes in Computer Science, vol. 14385. Springer, 2023, pp. 395–410.

[4] T. Olas, L. Szustak, R. Wyrzykowski, M. Olas, and M. Lapegna, "Advances in adapting memory-bound CFD computations to RISC-V multicore architecture," in *Computational Science – ICCS 2025*, ser. Lecture Notes in Computer Science, vol. 15905. Springer, 2025, pp. 151–166.

[5] Espressif Systems, *ESP32-C6 Series Datasheet*, Espressif Systems, 2024. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-c6_datasheet_en.pdf

[6] F. Bellard, "QEMU, a fast and portable dynamic translator," in *Proceedings of the 2005 USENIX Annual Technical Conference, FREENIX Track*. Anaheim, CA: USENIX Association, 2005, pp. 41–46.